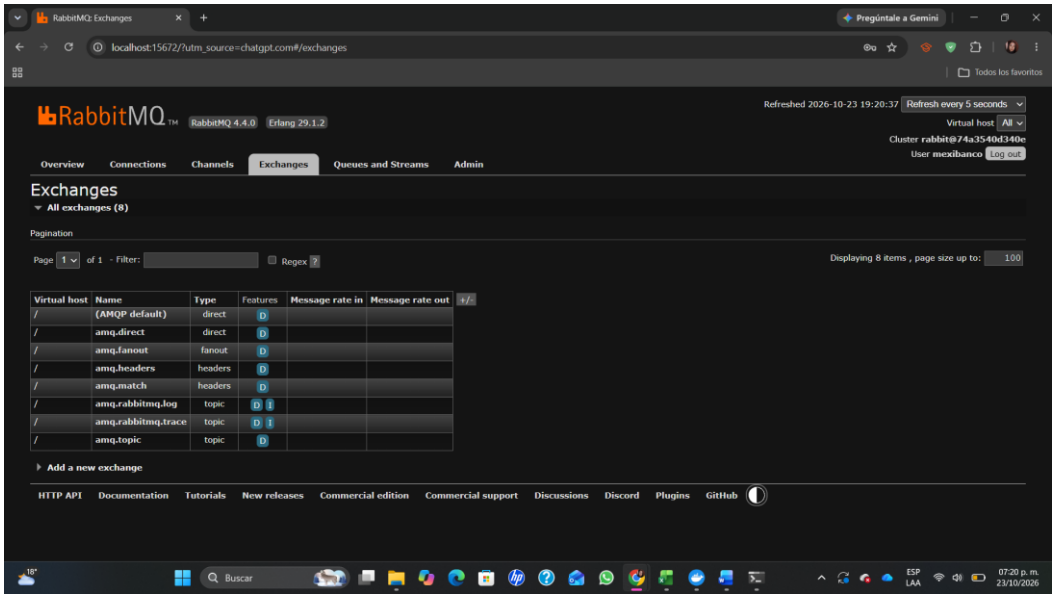
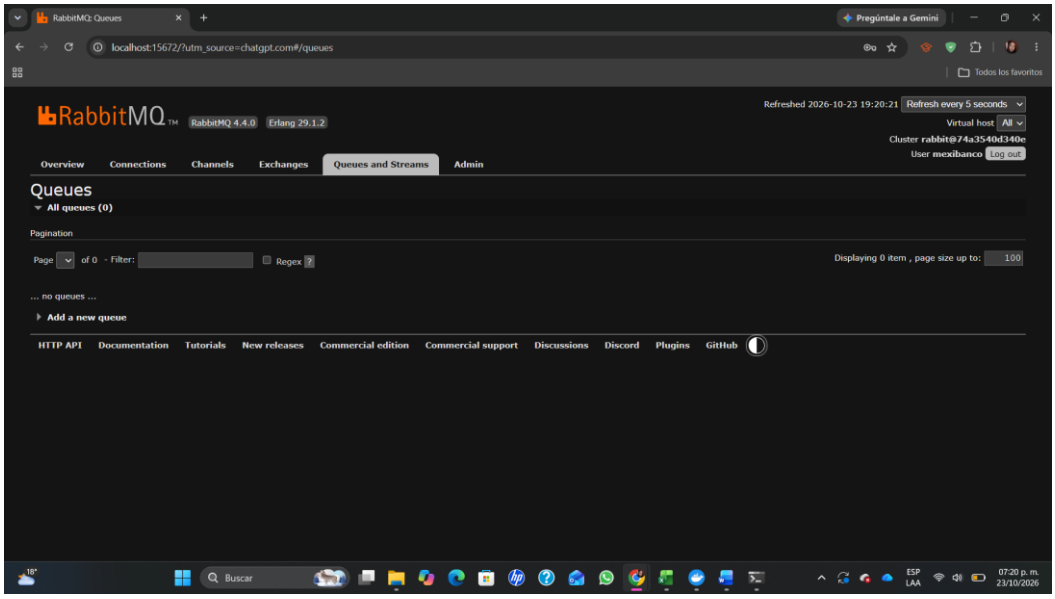
## 1. Situación inicial: acoplamiento HTTP

Primero revisé el panel de RabbitMQ: todavía no había una cola para las transferencias. Creé las cuentas de Ana, con $1,000, y Beto, con $500. Después detuve el servicio notificacion e intenté transferir $50.




*RabbitMQ – Exchanges.*




*RabbitMQ – Queues and Streams.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod -Uri "http://localhost:808
0/api/cuentas" -Method Post -ContentType "application/json" -Body '{"clabe":"002180000000000001","titular":"Ana","
saldoInicial":1000}'

id clabe              titular saldo
-- -----              ------- -----
 1 002180000000000001 Ana      1000


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod -Uri "http://localhost:808
0/api/cuentas" -Method Post -ContentType "application/json" -Body '{"clabe":"002180000000000002","titular":"Beto",
"saldoInicial":500}'

id clabe              titular saldo
-- -----              ------- -----
 2 002180000000000002 Beto      500


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09>
```

*Creación de la cuenta de Ana y Beto.*

La solicitud respondió HTTP 500 y la transferencia quedó en estado ABONADA, aunque el cargo y el abono ya habían ocurrido. Ana terminó con $950 y Beto con $550. Esto mostró el problema del aviso síncrono: una falla del servicio de notificaciones afectaba la respuesta y el estado final de una operación bancaria que ya había movido dinero.

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> try {
>>     $respuesta = Invoke-WebRequest `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -Method Post `
>>         -ContentType "application/json" `
>>         -Body '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}' `
>>         -UseBasicParsing
>>
>>     Write-Host "Codigo HTTP:" $respuesta.StatusCode
>>     Write-Host "Respuesta:" $respuesta.Content
>> }
>> catch {
>>     $respuesta = $_.Exception.Response
>>
>>     if ($null -ne $respuesta) {
>>         Write-Host "Codigo HTTP:" ([int]$respuesta.StatusCode)
>>
>>         $lector = New-Object System.IO.StreamReader($respuesta.GetResponseStream())
>>         Write-Host "Respuesta:" $lector.ReadToEnd()
>>         $lector.Close()
>>     }
>>     else {
>>         Write-Host "Error:" $_.Exception.Message
>>     }
>> }
Codigo HTTP: 500
Respuesta: {"timestamp":"2026-10-24T01:25:53.496+00:00","status":500,"error":"Internal Server Error","path":"/tran
sferencias"}
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09>
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09>
```

*Transferencia error.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod -Uri "http://localhost:808
0/api/transferencias?estado=ABONADA" -Method Get


id                    : 1
claveOrigen           : 002180000000000001
claveDestino          : 002180000000000002
monto                 : 50.00
fecha                 : 2026-10-24T01:25:51.436367Z
estado                : ABONADA
motivo                :
saldoOrigenTrasCargo  : 950.00
intentosDeCompensacion : 0
actualizada           : 2026-10-24T01:25:52.829509Z
```

*Transferencia abonada.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod -Uri "http://localhost:808
0/api/cuentas/002180000000000001" -Method Get

id clabe              titular  saldo
-- -----              -------  -----
 1 002180000000000001 Ana     950.00


PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod -Uri "http://localhost:808
0/api/cuentas/002180000000000002" -Method Get

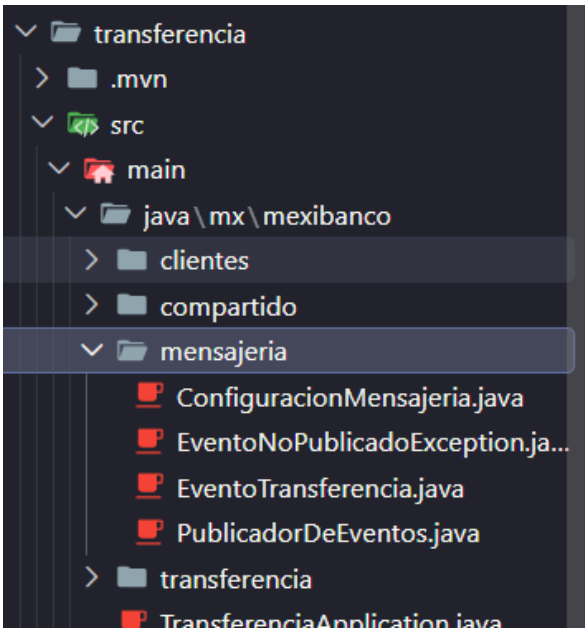
id clabe              titular  saldo
-- -----              -------  -----
 2 002180000000000002 Beto    550.00
```

*Saldos después del error.*

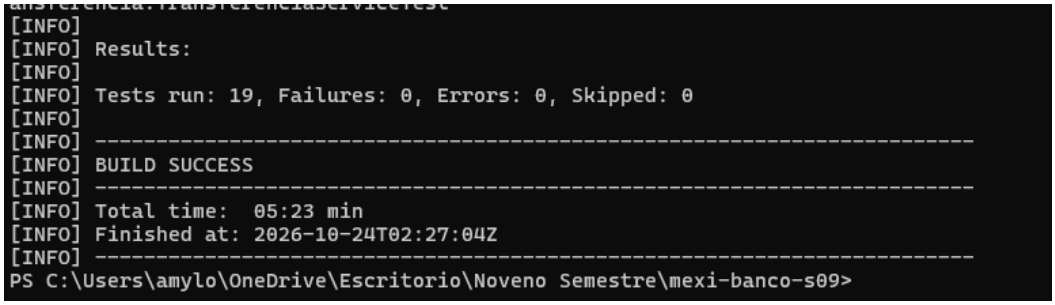
**2. Publicación asíncrona de eventos**

En transferencia agregué los contratos EventoTransferencia, ConfiguracionMensajeria, PublicadorDeEventos y EventoNoPublicadoException. Los eventos TransferenciaRealizada y TransferenciaRevertida llevan un identificador eventoId y se publican en el exchange topic durable transferencias, con routing keys transferencia.realizada y transferencia.revertida.

El publicador utiliza mensajes persistentes, publisher confirms y la opción mandatory para detectar eventos sin destino. Publisher confirms confirma la aceptación del mensaje por el broker, pero no garantiza que exista un consumidor, que el aviso quede almacenado en la base de notificacion ni que el cliente lo haya leído. Tampoco reemplaza un outbox transaccional.

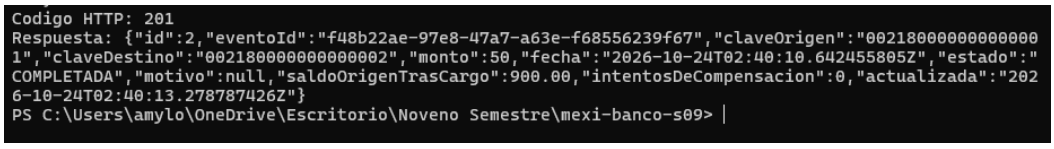



*Publicador eventos.*

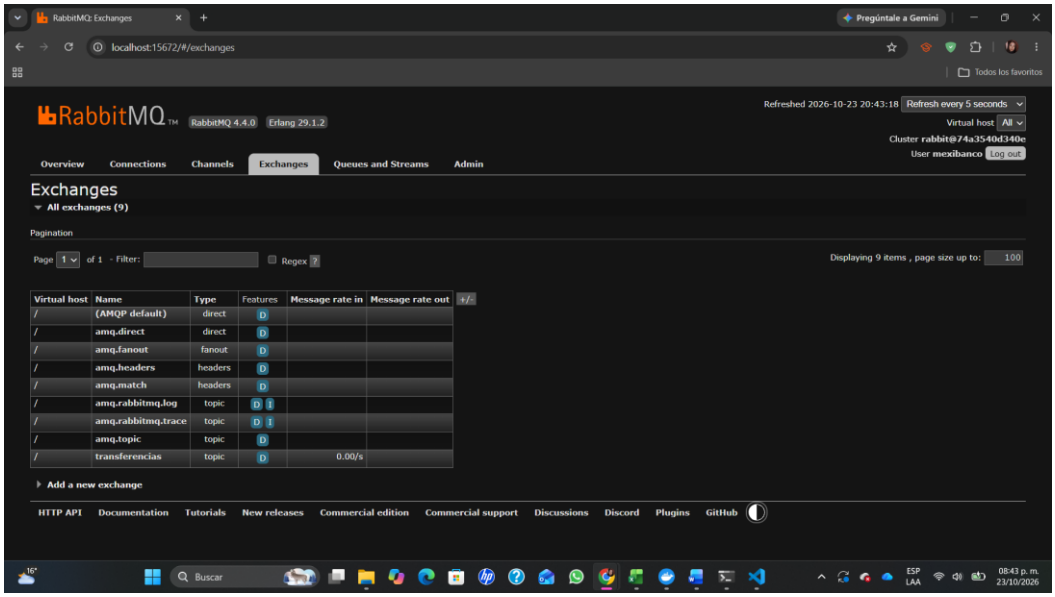



*Pruebas transferencia.*

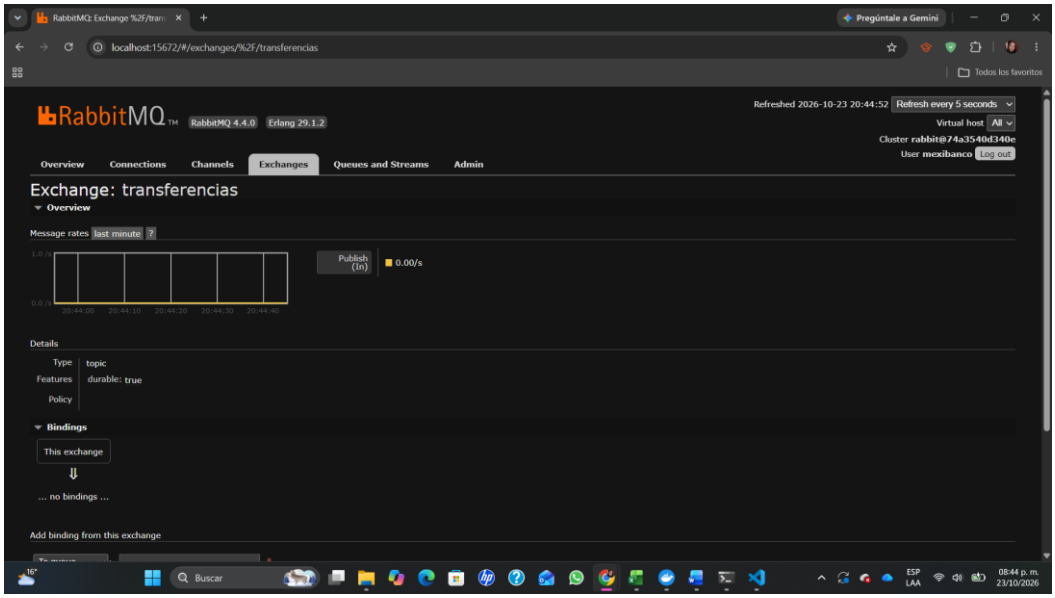
Al repetir la transferencia de $50 con notificacion apagado, obtuve HTTP 201 y estado COMPLETADA. Como aún no estaba enlazada la cola, el registro reportó NO_ROUTE; sin embargo, la transferencia ya no quedó detenida por el aviso. Este resultado demuestra el desacoplamiento, pero también un límite importante: en ese momento el evento no llegó a ningún consumidor.




*Transferencia completada.*




*Exchange Transferencias.*




*Exchange sin cola.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> docker compose -p mexi-banco-s0
9 logs --tail=100 transferencia | Select-String -Pattern "evento|public"

transferencia-1  | 2026-10-24T02:40:13.457Z ERROR 1 --- [transferencia] [nio-8080-exec-8]
m.m.transferencia.TransferenciaService    : Transferencia 2: quedo terminada pero el evento
TransferenciaRealizada f48b22ae-97e8-47a7-a63e-f68556239f67 NO se publico: Ninguna cola recibio el
evento f48b22ae-97e8-47a7-a63e-f68556239f67 con la routing key transferencia.realizada (NO_ROUTE)
```

*Evento sin cola.*

**3. Cola durable, consumidor y ACK manual**

En notificacion declaré la cola durable notificacion.transferencias y la vinculé al exchange transferencias con el patrón transferencia.*. El consumidor recibe el JSON, lo convierte a EventoTransferencia, guarda los avisos en PostgreSQL y después envía basicAck. Se configuró acknowledge-mode manual y prefetch 5.

López Hernández Miriam Amisadai

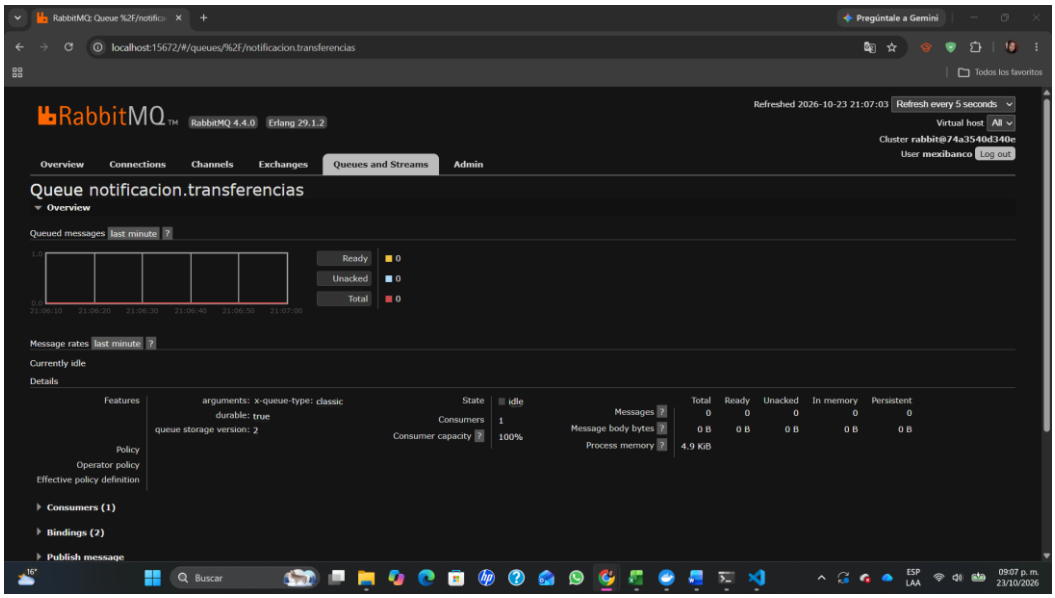
Las pruebas automatizadas de transferencia y notificacion terminaron correctamente: 19 pruebas en el primero y 1 prueba en el segundo, sin fallos. Los contenedores se reconstruyeron con los JAR actualizados y notificacion volvió a estado healthy.

```
ificacion.NotificacionServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:07 min
[INFO] Finished at: 2026-10-24T02:57:42Z
[INFO] ------------------------------------------------------------------------
```

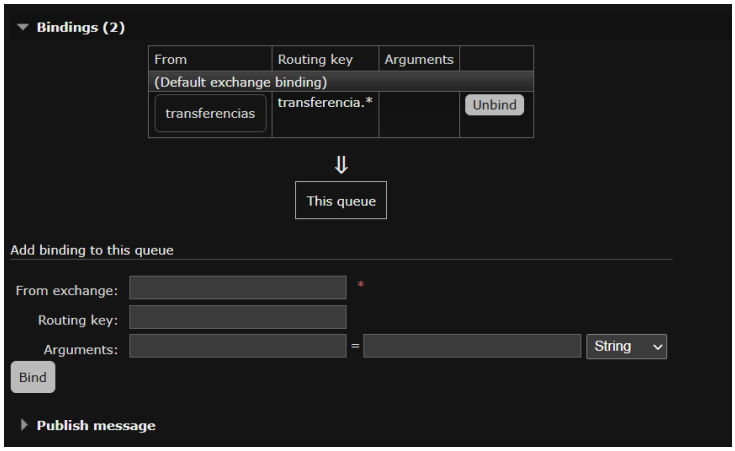
*Pruebas notificación.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> docker compose -p mexi-banco-s09
 ps notificacion
NAME                                IMAGE                                   COMMAND                  SERVICE
   CREATED              STATUS                          PORTS
mexi-banco-s09-notificacion-1   mexi-banco-s09-notificacion:local   "java -jar app.jar"   notificacion
   About a minute ago   Up About a minute (healthy)   8080/tcp
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09>
```

*Notificacion Healthy.*



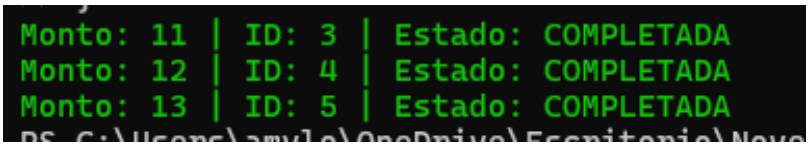

*Cola Durable.*



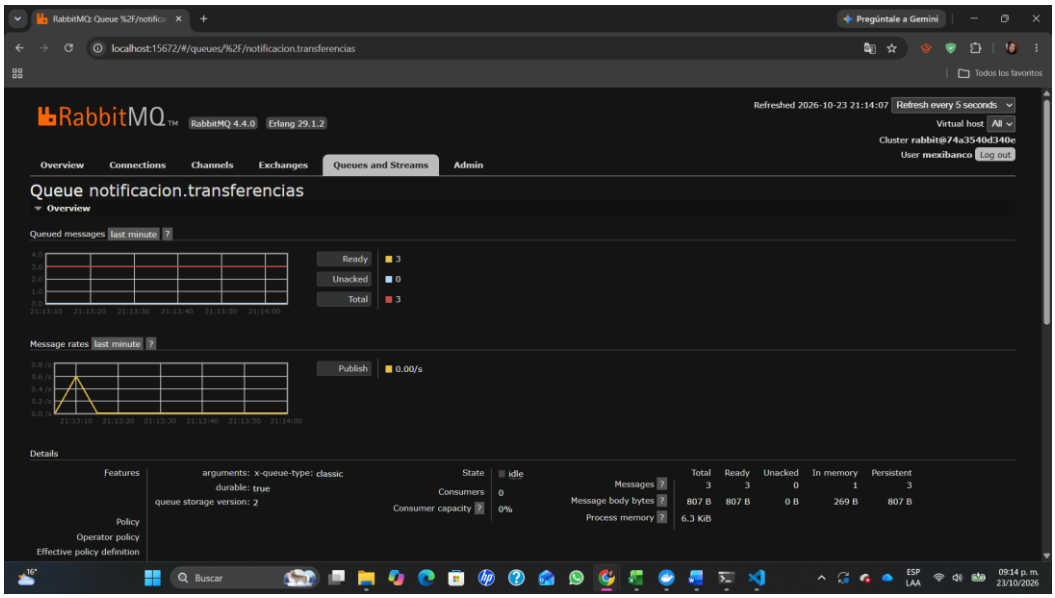

*Bindings transferencia.*

López Hernández Miriam Amisadai

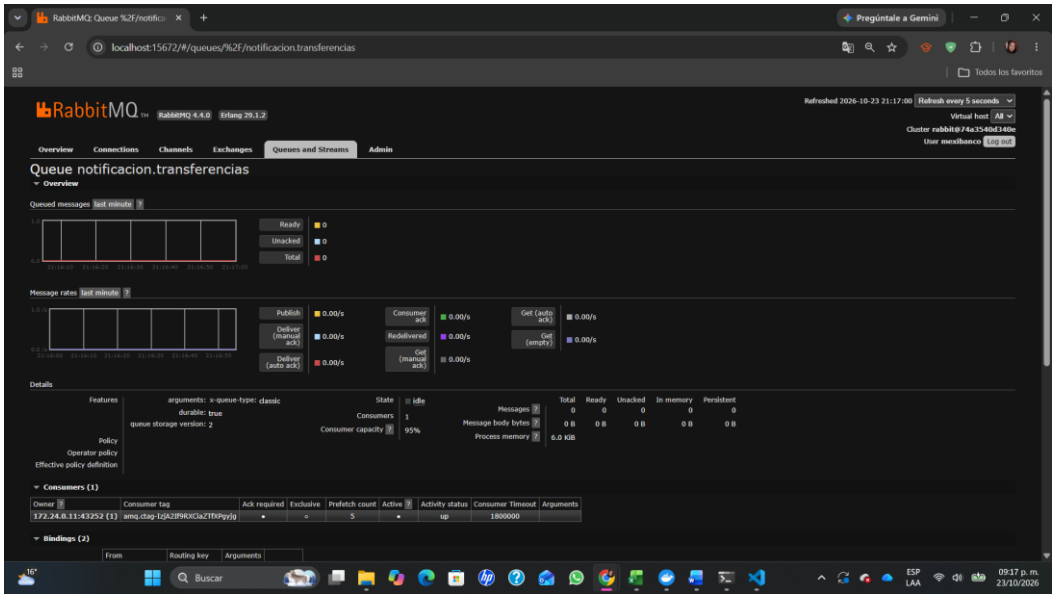
Detuve notificacion y realicé tres transferencias por $11, $12 y $13. Las tres terminaron en estado COMPLETADA; con el consumidor apagado la cola mostró Ready=3, Unacked=0 y Total=3. Al iniciar nuevamente notificacion, la cola descendió a cero y los registros mostraron dos avisos nuevos por cada evento.




*Tres transferencias.*




*Tres mensajes ready.*




*Cola procesada.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> docker compose -p mexi-banco-s09
 logs --tail=150 notificacion | Select-String -Pattern "consumidor|TRANSFERENCIA"

notificacion-1  | 2026-10-24T03:15:41.125Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000001 ┬À
TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00
notificacion-1  | 2026-10-24T03:15:41.126Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000002 ┬À
TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00
notificacion-1  | 2026-10-24T03:15:41.523Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 3
(evento c2c83a0b-12f2-40a6-b346-edd1a715db2e, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T03:15:41.532Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000001 ┬À
TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00
notificacion-1  | 2026-10-24T03:15:41.532Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000002 ┬À
TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00
notificacion-1  | 2026-10-24T03:15:41.546Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 4
(evento 622487d1-b859-44fe-8210-b7f7eb612feb, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T03:15:41.549Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000001 ┬À
TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00
notificacion-1  | 2026-10-24T03:15:41.549Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000002 ┬À
TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00
notificacion-1  | 2026-10-24T03:15:41.563Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 5
(evento eaf4d7e0-228c-41de-ad11-30b1655af57f, redelivered=false): 2 aviso(s) nuevo(s)
```

*Eventos consumidos.*

Consulté las bandejas de Ana y Beto: Ana recibió TRANSFERENCIA_ENVIADA y Beto TRANSFERENCIA_RECIBIDA para los montos $11, $12 y $13. En total se guardaron seis avisos. De esta manera comprobé la persistencia en la base del consumidor, además de los contadores del broker.

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod `
>>   -Uri "http://localhost:8080/api/notificaciones?clabe=002180000000000001" `
>>   -Method Get | Format-Table -AutoSize

id clabeCuenta        mensaje                                                fecha
-- -----------        -------                                                -----
 5 002180000000000001 TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00 2026-10-24T03:15:41.54...
 3 002180000000000001 TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00 2026-10-24T03:15:41.53...
 1 002180000000000001 TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00 2026-10-24T03:15:41.12...
```

*Notificaciones Ana.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> Invoke-RestMethod `
>>   -Uri "http://localhost:8080/api/notificaciones?clabe=002180000000000002" `
>>   -Method Get | Format-Table -AutoSize

id clabeCuenta        mensaje                                                fecha
-- -----------        -------                                                -----
 6 002180000000000002 TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00 2026-10-24T03:15:41.5...
 4 002180000000000002 TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00 2026-10-24T03:15:41.5...
 2 002180000000000002 TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00 2026-10-24T03:15:41.1...
```

*Notificaciones Beto.*

**Compensación**

Simulé una falla de abono hacia la CLABE de Beto con FALLAR_ABONO_A_CLABE. La transferencia de $150 respondió HTTP 502 y terminó COMPENSADA: se devolvió el dinero al origen. El evento TransferenciaRevertida fue consumido y generó un aviso DEVOLUCION_TRANSFERENCIA para Ana. Después restauré la configuración para desactivar la falla simulada.

```
Codigo HTTP: 502
Respuesta: {"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pud
o abonar a la CLABE 002180000000000002, asi que se devolvieron 150 a la CLABE 002180000000000001: su s
aldo quedo como estaba.","instance":"/transferencias","transferencia":6,"estado":"COMPENSADA","motivo"
:"El abono a la CLABE 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=0021800000000
00002"}
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09>
```

*Transferencia compensada.*

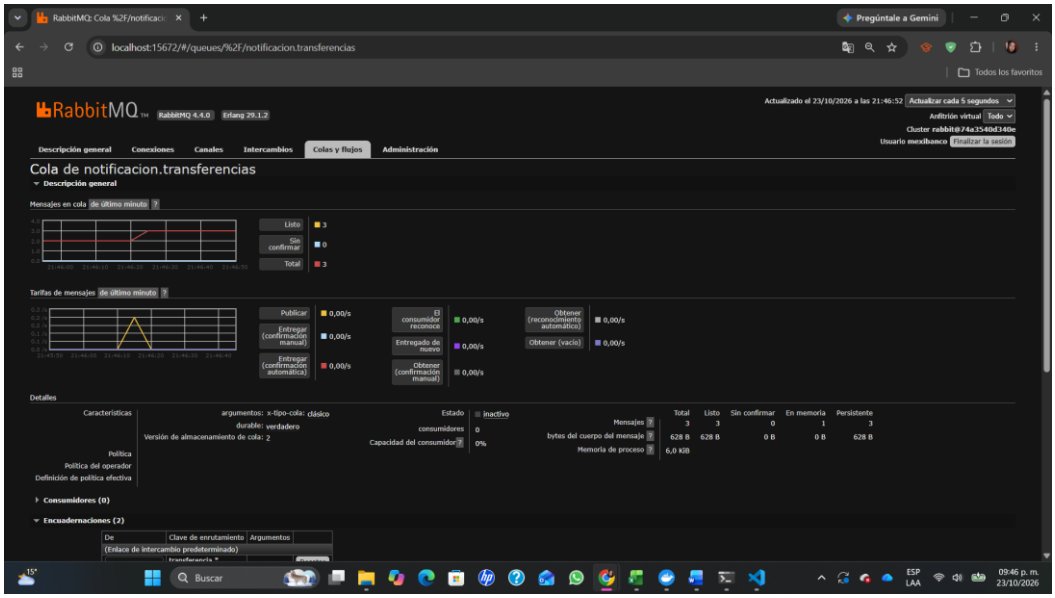
```
notificacion-1  | 2026-10-24T03:15:41.523Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 3
(evento c2c83a0b-12f2-40a6-b346-edd1a715db2e, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T03:15:41.546Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 4
(evento 622487d1-b859-44fe-8210-b7f7eb612feb, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T03:15:41.563Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 5
(evento eaf4d7e0-228c-41de-ad11-30b1655af57f, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T03:26:10.150Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 002180000000000001 ┬À
DEVOLUCION_TRANSFERENCIA por 150, saldo resultante 864.00
notificacion-1  | 2026-10-24T03:26:10.196Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRevertida de la transferencia 6
(evento 75f58900-3665-4cf8-be6b-0ffb099aaac6, redelivered=false): 1 aviso(s) nuevo(s)
```

*Evento revertido.*

**4. Idempotencia frente a eventos duplicados**

RabbitMQ puede volver a entregar un mensaje si se pierde la confirmación. Sin una validación, el mismo evento produciría un segundo juego de avisos. Implementé la entidad EventoProcesado, con eventoId como clave primaria, y una inserción PostgreSQL INSERT ... ON CONFLICT DO NOTHING. El registro del evento y el guardado de avisos ocurren en una misma transacción.

Publiqué dos veces el evento simulado con el mismo eventoId. El primer mensaje quedó registrado y produjo dos avisos; el segundo fue reconocido como duplicado y produjo cero avisos nuevos. La captura de la cola muestra tres pendientes porque además había un mensaje inválido; los registros confirman la prueba del duplicado. No se adjunta una medición anterior al cambio de idempotencia, por lo que la comparación con el comportamiento anterior es explicativa y no una evidencia experimental separada.



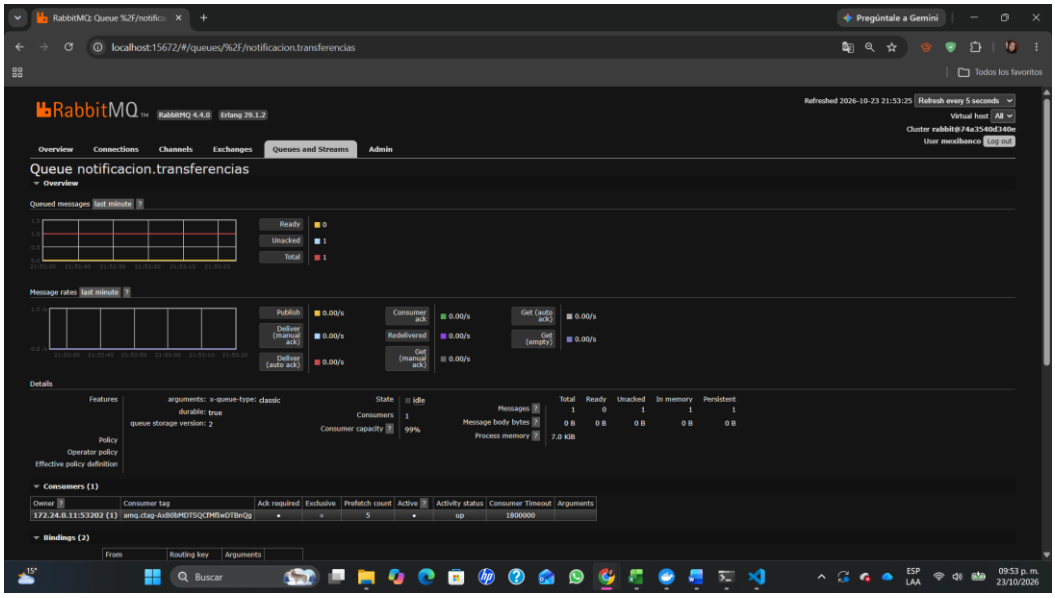

*Eventos Pendientes Idempotencia.*

```
> notificacion-1  | 2026-10-24T03:49:18.296Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [idempotencia] Evento
75f58900-3665-4cf8-be6b-0ffb099aaac6 registrado; 2 aviso(s) guardado(s)
> notificacion-1  | 2026-10-24T03:49:18.328Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia
999 (evento 75f58900-3665-4cf8-be6b-0ffb099aaac6, redelivered=false): 2 aviso(s) nuevo(s)
> notificacion-1  | 2026-10-24T03:49:18.342Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [idempotencia] Evento
75f58900-3665-4cf8-be6b-0ffb099aaac6 duplicado; no se crean avisos
> notificacion-1  | 2026-10-24T03:49:18.344Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia
999 (evento 75f58900-3665-4cf8-be6b-0ffb099aaac6, redelivered=false): 0 aviso(s) nuevo(s)
  notificacion-1  | 2026-10-24T03:49:18.752Z  INFO 1 --- [notificacion] [nio-8080-exec-1]
o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
  notificacion-1  | 2026-10-24T03:49:18.753Z  INFO 1 --- [notificacion] [nio-8080-exec-1]
o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
```

*Idempotencia sin Duplicados.*

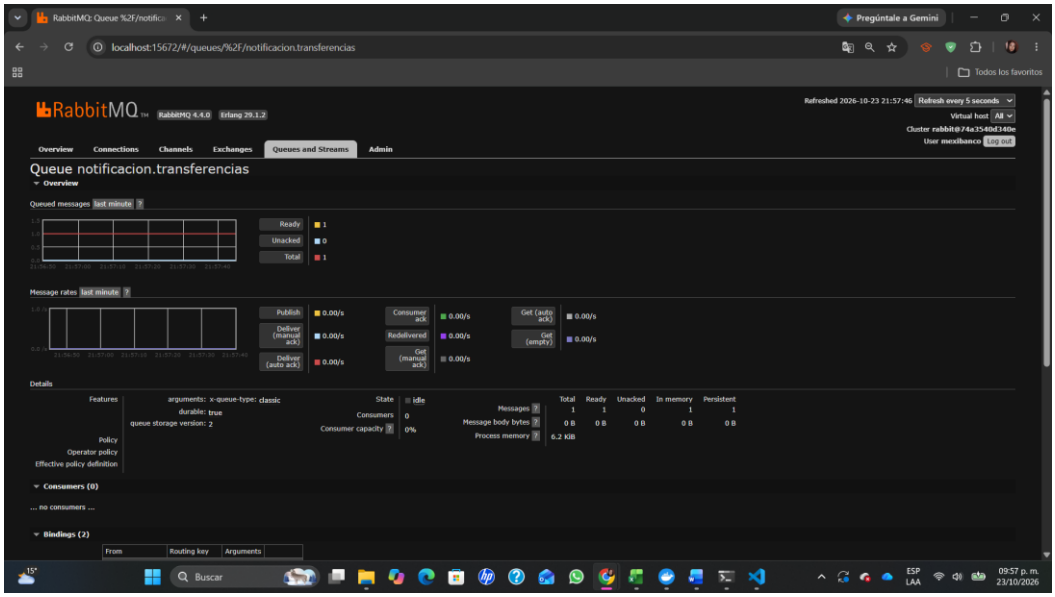
**5. Mensajes inválidos y cola de errores (DLQ)**

Durante la prueba apareció un mensaje sin JSON legible. Antes de implementar la DLQ provocaba MismatchedInputException y quedaba sin confirmar. Al detener el consumidor, ese mensaje regresó a Ready. Guardé la evidencia antes de recrear la cola con los argumentos de dead lettering.

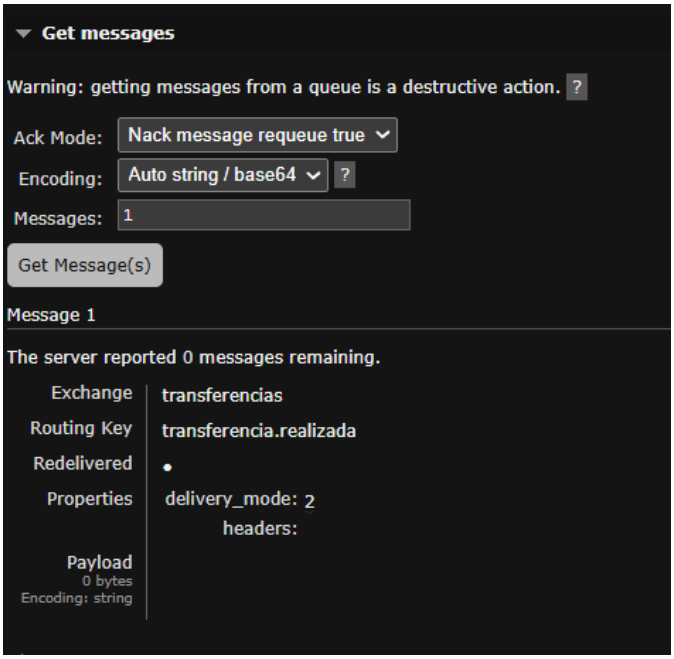



*Mensaje Atascad sin DLQ.*

López Hernández Miriam Amisadai



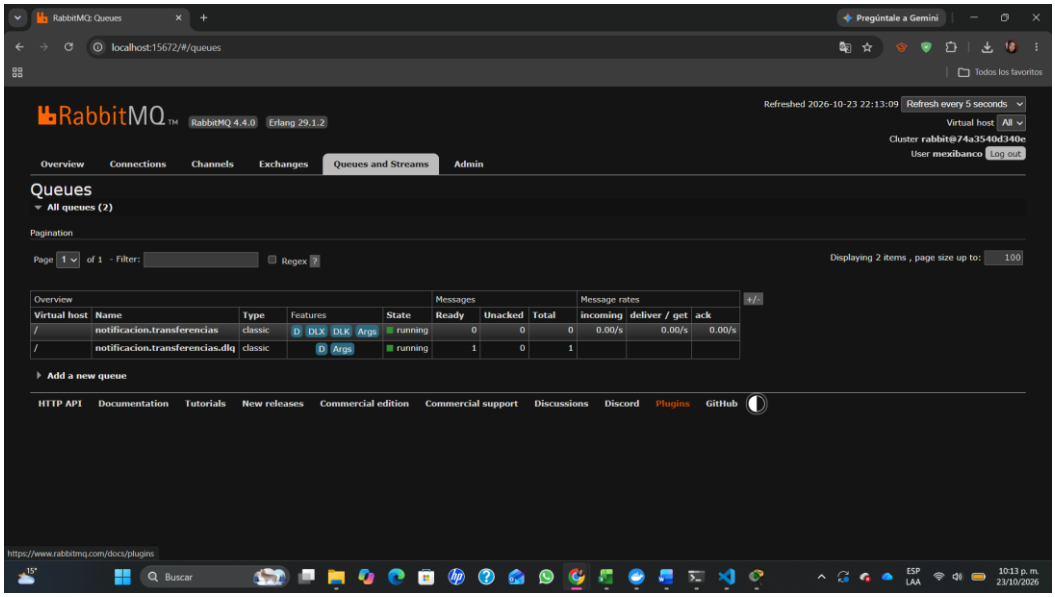

*Mensaje pendiente antes de DLQ.*



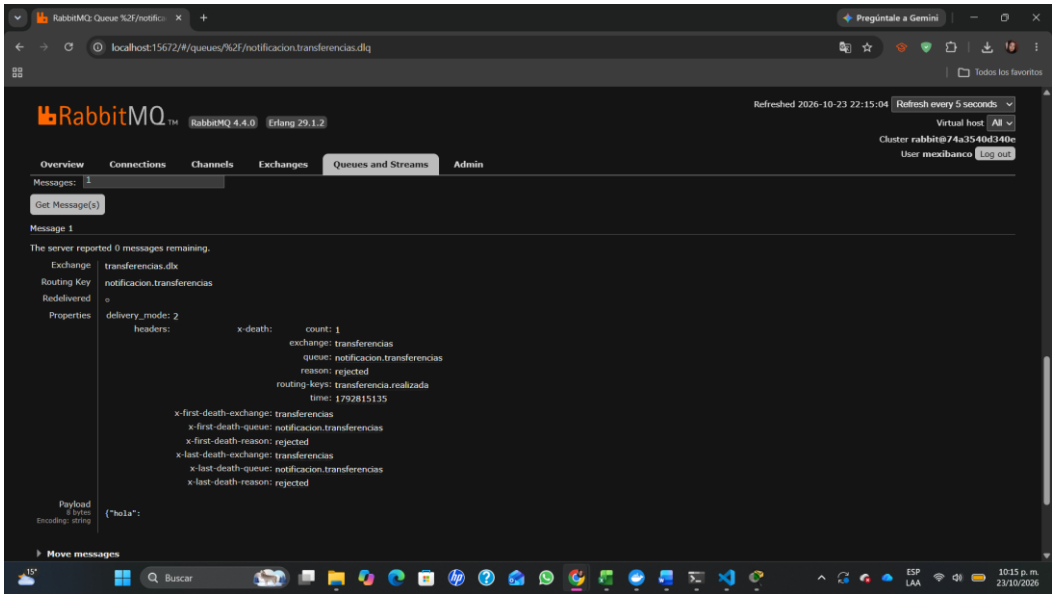

*Mensaje invalido.*

Después declaré el exchange directo durable transferencias.dlx y la cola durable notificacion.transferencias.dlq. Para JSON inválido, el consumidor ejecuta basicNack(deliveryTag, false, false), por lo que RabbitMQ lo aparta de la cola principal. Probé con el contenido incompleto {"hola": y observé cero mensajes en la cola principal y uno en la DLQ.

López Hernández Miriam Amisadai

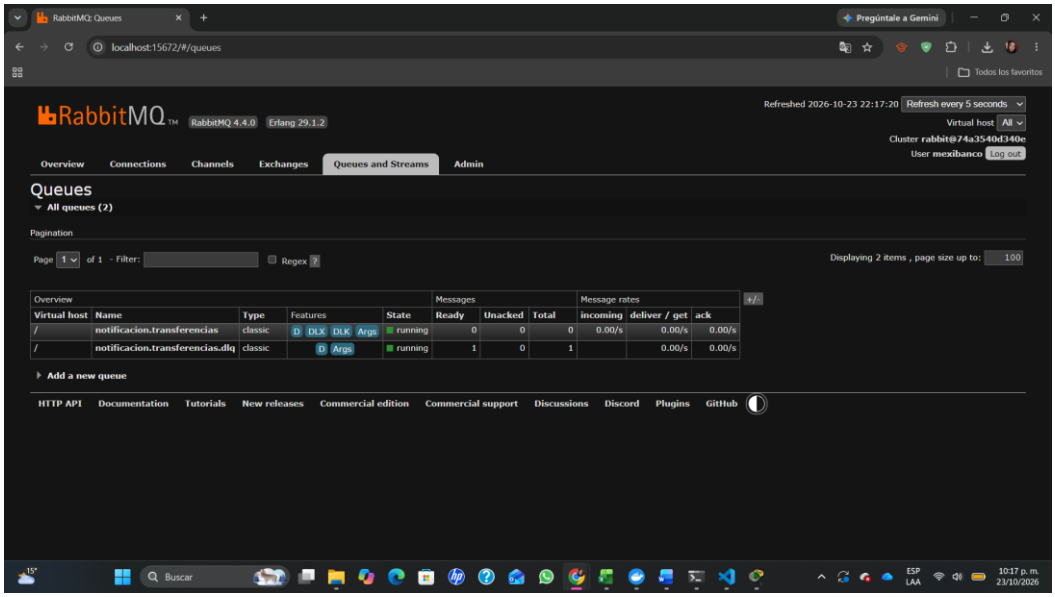



*Mensajes en DLQ.*




*DLQ_XDeath_Rejected.*

Al inspeccionar el mensaje en la DLQ confirmé el encabezado x-death con reason=rejected y count=1. Finalmente publiqué un evento de prueba válido con ID de transferencia 1000 y el servicio registró dos avisos nuevos, mientras la DLQ conservó su mensaje inválido. Esta prueba demuestra que un mensaje defectuoso ya no bloquea la continuación del procesamiento normal.




*Consumidor después de DLO.*

```
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\mexi-banco-s09> docker compose -p mexi-banco-s09
 logs --tail=100 notificacion |
>>     Select-String -Pattern "ed722d75|1000|idempotencia"

notificacion-1  | 2026-10-24T04:16:34.361Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [idempotencia] Evento
ed722d75-4baf-4b07-9c23-7be9cb93a8f1 registrado; 2 aviso(s) guardado(s)
notificacion-1  | 2026-10-24T04:16:34.463Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de transferencia 1000
(evento ed722d75-4baf-4b07-9c23-7be9cb93a8f1): 2 aviso(s) nuevo(s)
```

*Consumidor funcionando.*

## 6. Preguntas de reflexión

**¿Qué sucede si RabbitMQ falla cuando la saga ya está COMPLETADA?**
Como el aviso ya no es parte de la operación bancaria principal, la transferencia puede quedar COMPLETADA sin que se haya entregado la notificación. Los publisher confirms ayudan a saber si RabbitMQ aceptó una publicación, pero no hacen atómica la actualización de PostgreSQL y la publicación del mensaje. Si el broker está caído o no hay ruta a una cola, el evento podría perderse; una mejora futura sería usar un outbox transaccional para publicar después los eventos pendientes.

**¿Por qué se envía ACK después de guardar los avisos?**
Porque RabbitMQ debe considerar terminado el trabajo solo cuando los datos ya fueron persistidos. Si enviáramos ACK antes y luego fallara PostgreSQL, el broker quitaría el mensaje aunque faltaran los avisos. Al confirmar después del guardado, una falla antes del ACK permite reentregarlo. Esa reentrega exige idempotencia, porque los avisos podrían haberse guardado antes de perder la confirmación.

**¿Qué se gana y qué se sacrifica al cambiar de HTTP síncrono a mensajería asíncrona?**
Se gana independencia entre los servicios: la transferencia puede concluir aunque notificacion no esté disponible, y una cola durable conserva eventos hasta que haya un consumidor. A cambio ya no se puede prometer que el aviso llegará inmediatamente ni usar su respuesta para decidir el resultado de la transferencia. Los datos se actualizan de forma eventual y se necesita vigilar el broker, los reintentos, los duplicados y la DLQ.

## 7. Conclusión

En esta práctica comprobé que una transferencia no debe depender de que el aviso salga bien. Con RabbitMQ logré completar la operación y guardar los eventos cuando notificacion estuvo apagado. Después incorporé ACK manual, idempotencia para no repetir avisos y DLQ para separar mensajes inválidos. La solución mejora la disponibilidad de la operación principal, aunque exige aceptar y controlar la entrega eventual de las notificaciones